Vishwakarma Institute of Technology, Pune-411037, Maharashtra, India

CS3111 Web Technologies · Computer Engineering · AY 2026-27

System Requirements and Architecture Specification

PS Laser — Laser-Cutting Job-Shop Web Platform

Prepared by
Rahul Diwate
Rivan Shetty
Gaurav Shende
Vishal Sharma
Gandharv Shinde

Version 1.0 (draft for faculty review)
18 October 2026

Document control

Item	Value
Companion document	IEEE-format research paper "PS Laser: A Multi-Stack Web Platform …"
Basis	Project repository (D:\WT CP PS LASER), `services/CONTRACT.md`, operations handbook, QA run of 14 Oct 2026, re-checks on 16 and 18 Oct 2026
Standards followed	Section structure guided by ISO/IEC/IEEE 29148:2018; quality attributes per ISO/IEC 25010:2011
Status tags	**[V]** verified (run) · **[S]** from source/documents, not run here · **[P]** proposed or assumed, not measured

Contents

1. Project Overview
2. Functional Requirements
3. Non-Functional Requirements
4. Hardware Requirements
5. Software Requirements
6. Technology Stack
7. System Architecture
8. Architecture Diagram
9. Data Flow and Workflow
10. Interfaces and Dependencies
11. Installation and Deployment Requirements
12. Assumptions, Constraints and Future Enhancements
References

1. Project Overview

1.1 Purpose

PS Laser is a sheet-metal and CNC laser-cutting job shop at Gat No. 753, Velu, Tal. Bhor, Dist. Pune 412205. This document specifies the requirements and architecture of the web platform built for the shop: a public website, an instant indicative cost estimator, enquiry and drawing capture, order tracking, and staff back-office tools. The platform is also the course project for CS3111 *Web Technologies*, so each syllabus unit is implemented as a real feature of the same business.

1.2 Scope

In scope: the seven software services listed in Section 7, the shared MySQL database, the optional MongoDB activity log, and the local run environment on a Windows PC.

Out of scope: integration with shop-floor machines (the lasers, press brake, turret punches, welder and coating plant are described for context only), payment processing, e-mail/SMS delivery, public internet deployment, and the syllabus lab practicals (electricity bill, book store and similar), which are intentionally not part of this project.

1.3 Objectives

- Present the shop's real machines, services and contact channels on a responsive website.
- Give customers an instant, clearly labelled **indicative** price from a rate card the owner controls.
- Capture enquiries and drawings reliably and let staff manage them.
- Let customers track a job to dispatch with an order number and phone number.
- Keep seven independently built services consistent through one schema and one contract.

1.4 Users

Table 1. User classes

User	What they do
Customer	Browses services, gets an estimate, submits an enquiry with a drawing, tracks an order, opens WhatsApp for a final quote
Staff / owner	Reviews enquiries, maintains machines, services and the rate card, creates work orders and updates their status
Administrator / developer	Runs and configures the services, seeds the database, manages credentials and secrets

1.5 Business facts the system is built on

Seven facilities are described, from the company profile: HSG 3 kW fiber laser, Genesis 1.5 kW fiber laser (both 3 m × 1.5 m beds), 175 t CNC press brake (3 m), Amada 32 t and 16 t CNC turret punches, HSG 1.5 kW handheld laser welder and a PLC powder-coating plant. Laser-cutting capacity is mild steel 16 mm, stainless steel 12 mm, aluminium 8 mm, brass 4 mm and copper 4 mm. The estimator's **price rates are sample values** (database flag `rates.is_sample = 1`); machines, services and contact data are real.

2. Functional Requirements

Table 2 lists what the system must do. "Where" names the service that provides the function; "Status" uses the tags defined on the cover page.

Table 2. Functional requirements

ID	Requirement	Where	Status
FR-01	Public site with Home, About, Engineering, Architectural, Gallery, Pattern Design, Blog and Contact pages; unknown URLs return 404	Next.js	[V] 14 Oct

ID	Requirement	Where	Status
FR-02	Catalogue of machines, services and materials read from the database	Spring, Node	[V] 14 Oct
FR-03	Indicative estimate from material, thickness, length, width, cut length, pierces, bends, powder-coat flag and quantity; breakdown plus GST; result labelled "Indicative"	Spring, Next.js, PHP, JSP	[V] 16 Oct (Spring); [V] 14 Oct (all four)
FR-04	Validate estimate inputs (material exists, thickness within limit and inside a rate band, part within 3000 × 1500 mm, quantity ≥ 1); reject with HTTP 400 and field errors	Spring, Next.js	[V] 14 Oct (MS 20 mm → 400)
FR-05	Offline estimate in the browser stack when the API is unreachable, labelled "Offline estimate (sample)"	Next.js	[S]
FR-06	Save a quote and return a quote number (`PSL-Q-YYYYMMDD-####`)	Spring	[S]
FR-07	Accept an enquiry (name, phone, message, optional e-mail, service, machine) with an optional drawing (.dxf .dwg .pdf .png .jpg, ≤ 10 MB); return an id and a WhatsApp link	Node, PHP	[V] by Node suite 18 Oct
FR-08	Every "Get Quote" action opens `wa.me/917719976990` with a pre-filled, context-specific message	All front ends	[S]
FR-09	Order tracking by order number and phone number; wrong phone returns "not found"; phone shown masked; timeline of events	Spring, Next.js, JSP	[V] 14 Oct
FR-10	Customer registration, login, profile and "my quotes"	Spring, React portal	[V] login 14 Oct; [S] register
FR-11	Staff/admin login with roles admin, staff and customer; admin endpoints reject requests without a valid token	Spring, Node, Laravel, JSP	[V] 14 Oct
FR-12	Staff create and update work orders through seven stages (received, cutting, bending, welding, coating, inspection, dispatched), each change recorded as an event	Spring, Laravel, JSP	[V] by Laravel suite 18 Oct; [S] Spring, JSP
FR-13	Live work-order dashboard updated by a server push	Spring (STOMP), Next.js	[S]; push not verified end to end
FR-14	Back-office maintenance of machines, services, rate card, enquiries and quotes; staff cannot delete	Laravel	[V] 7 screens 14 Oct; [V] 8 tests 18 Oct
FR-15	Staff list enquiries, change enquiry status and download a drawing	Node, Spring, Laravel	[V] by Node suite 18 Oct
FR-16	Record audit events (logins, estimates, status changes) in an activity log	Spring → MongoDB	[V] health only (16 Oct)
FR-17	Health and monitoring endpoints and an API documentation page	Spring, Node	[V] 16, 18 Oct

3. Non-Functional Requirements

Targets marked [P] are proposals for the faculty and owner to confirm; **no performance measurement has been taken**, so none is claimed.

Table 3. Non-functional requirements

Attribute	Requirement	How addressed / evidence
Performance	An estimate should return fast enough to feel instant on a local network [P: under 1 s]; the estimate input is debounced	350 ms debounce in the calculator [S]; not benchmarked
Reliability	The public estimator and portal keep working when Spring is down; money values never differ between stacks	Offline fallbacks [S]; one rounding rule; same total on four stacks [V]

Attribute	Requirement	How addressed / evidence
Security	Hash passwords; sign tokens; protect forms and admin routes; validate input and uploads; limit abuse; avoid exposing private files	bcrypt `$2y$`; HS256 JWT; httpOnly cookie; CSRF (PHP, Laravel); zod and prepared statements (Node); upload type and size rules; rate limit of 20 enquiries per 15 min by default; helmet; CORS allow-list [S]; 401/403/419/303 behaviour [V]
Privacy	Tracking reveals an order only to someone who knows both order number and phone	Both required; phone masked [V]
Scalability	Services are stateless (JWT) and independently startable; one shared database is the bottleneck	Design property [S]; not load-tested
Usability	Mobile-first, readable on phones; clear "Indicative" labelling; WhatsApp as the primary call to action	Responsive CSS and sticky mobile total bar [S]; visual pass at 375/768/1440 px not done
Maintainability	Layered code, one written contract, documented mapping to the syllabus, automated tests, lint and type checks	Controller→service→repository in Spring; `CONTRACT.md`; Node 33 and Laravel 8 tests passed 18 Oct [V]; Spring has 15 test methods [S]; `npm run check` passed 14 Oct [V]
Portability	Runs on a Windows PC from documented commands; container image is optional	Verified on Windows 11 [V]; the Spring Dockerfile image was never built [S]
Observability	Health checks and logs for each service	Spring actuator, Node request/event logs, PHP estimate log, Laravel and Tomcat logs [S]
Accessibility	Respect reduced-motion preference	`prefers-reduced-motion` disables shimmer and transitions [S]

4. Hardware Requirements

4.1 Reference development machine [V]

Table 4. Machine on which the system was built and run

Item	Value
Processor	Intel Core i5-12450H, 8 cores / 12 threads
Memory	15.6 GB
Operating system	Windows 11 Home Single Language 10.0.26300
Storage note	Drive C: was 100 % full; the project was therefore moved to drive D:. Build artefacts (Next.js cache, node_modules, Laravel vendor, Maven cache, Spring target) together occupy roughly 1.8 GB, and about 100 MB more is downloaded for embedded MongoDB on first run

4.2 Minimum and recommended server/host specification [P]

All eight server processes (Spring and Tomcat on Java; Node ×2 including the Next.js dev server and portal; PHP ×2; MariaDB) run on one PC. The figures below are proposals derived from the reference machine and the folder sizes above; they have not been tested on smaller hardware.

Table 5. Host hardware

Component	Minimum [P]	Recommended [P]
Processor	4 cores, 64-bit x86	8 cores or more
RAM	8 GB	16 GB
Storage	10 GB free (SSD preferred)	20 GB free SSD, on a drive other than a full system drive
Display	1366 × 768	1920 × 1080 (for the dashboard and admin screens)

Component	Minimum [P]	Recommended [P]
Network	Loopback only for local use; internet for first-time installs, CDN assets and WhatsApp links	Broadband connection
Peripherals	Keyboard and pointing device	None required

4.3 Client devices [P]

Any smartphone, tablet or PC with a current Chrome, Edge, Firefox or Safari browser and a viewport of at least 360 px width. No special client hardware or plug-in is needed.

4.4 Project-specific devices

None. The software has **no hardware interface** to the shop's machines. For context, the physical equipment is: two fiber lasers (3 kW and 1.5 kW), a 175 t CNC press brake, Amada 32 t (40 stations) and 16 t (26 stations) turret punches, a 1.5 kW handheld laser welder and a PLC powder-coating plant with 9-tank pre-treatment. Staff read estimates and enquiries on screen and update job status manually.

5. Software Requirements

Table 6. Software used (versions recorded on the reference machine, 14–18 Oct 2026) [V]

Category	Software	Version	Purpose
Operating system	Windows	11 (10.0.26300)	Host OS (other OSs not tested)
JavaScript runtime	Node.js / npm	24.12.0 / 11.10.1 (`engines`: node ≥ 24)	Next.js, React portal, Node API
Java	Eclipse Adoptium JDK	21.0.12	Spring Boot, Maven wrapper, Tomcat
Bundled stack	XAMPP	PHP 8.2.12, MariaDB 10.4.32, Tomcat 8.5.96	PHP, Laravel, MySQL, Servlet/JSP
PHP packages	Composer	2.10.3	Laravel dependencies
Build	Maven wrapper (`mvnw`)	bundled in repo	Spring build and run
Containers (optional)	Docker Desktop	29.6.1 (engine was stopped)	Optional MongoDB container
Version control	Git	2.52	Source control
Browser	Edge / Chrome	current	Use and testing
API testing	Postman	collection in `services/spring-api/postman`	Spring API checks

5.1 Key libraries per service [S]

Table 7. Declared dependencies

Service	Main libraries
Next.js site	next 16.3.5, react/react-dom 19.2.4, zustand 5, bootstrap 4.5, tailwindcss 4, TypeScript 5, aos, owl.carousel
React portal	react 19.2, react-router-dom 7, @reduxjs/toolkit 2, react-redux 9, vite 8
Spring API	Spring Boot 3.5.16 (web, validation, data-jpa, data-mongodb, security, websocket, actuator), MariaDB JDBC driver, jjwt 0.12.6, embedded MongoDB, H2 (tests)
Node API	express 5.3, zod 4, multer 2.4, helmet 8, express-rate-limit 8, jsonwebtoken 9, mysql2 3, cors, dotenv
Laravel admin	laravel/framework $^{12.0 \text{ on PHP}}$ 8.2, tinker; SQLite in memory for tests
PHP estimator	No framework and no Composer; PDO; Bootstrap 4.5 and jQuery from CDN
Servlet/JSP	javax.servlet on Tomcat 8.5, JSTL, JDBC; built into `pslaser.war`

6. Technology Stack

Table 8. Technology stack by layer

Layer	Technologies
Frontend	Next.js 16 (App Router, server components, route handlers, server actions), React 19, TypeScript, Tailwind CSS 4, Bootstrap 4 (classic design), Zustand; React portal with Vite, Redux Toolkit and React Router; plain PHP/HTML templates; JSP views with JSTL
Backend	Spring Boot 3.5 (Java 21), Node.js 24 with Express 5, PHP 8.2, Laravel 12, Java Servlets/JSP on Tomcat 8.5
Database	MySQL/MariaDB 10.4 (database `pslaser`, eleven shared tables plus framework tables); MongoDB for the activity log (embedded or Docker)
APIs	REST over HTTP/JSON (Spring on 8081, Node on 4000, Next.js route handlers); STOMP over WebSocket (`/topic/work-orders`); `wa.me` deep links
Security	HS256 JWT with a shared secret; bcrypt; CSRF tokens; httpOnly cookie; CORS allow-list; zod validation; helmet; rate limiting
Languages	TypeScript/JavaScript, Java, PHP, SQL, HTML/CSS
Tooling	npm, Maven wrapper, Composer, ESLint, TypeScript compiler, JUnit/Mockito/MockMvc, node:test with supertest, PHPUnit, Postman

7. System Architecture

The system has five tiers: client, presentation, application/API, data, and (for context) the physical shop. Table 9 states the responsibility of each component; Section 8 shows the diagram.

Table 9. Components and responsibilities

Component (port)	Responsibility	Depends on
Next.js website (3001)	Public pages; /quote estimator; /track order tracking; /dashboard live board for staff; route handlers `/api/estimate`, `/api/auth/*`, `/api/work-orders`; keeps the JWT in an httpOnly cookie	Spring (optional), Node
React portal (5173)	Customer portal: catalogue, estimate, quote cart, login, my quotes, tracking; demo data if APIs are down	Spring, Node
PHP estimator (8000)	Stand-alone estimator form, drawing upload and enquiry form with CSRF protection	MySQL
Laravel admin (8001)	Staff back office: enquiries, machines, services, rate card, quotes, work orders	MySQL
Servlet/JSP desk (8080, /pslaser)	Customer order tracking and staff job-status desk using sessions, cookies and URL rewriting	MySQL
Spring Boot API (8081)	Main REST API: catalogue, estimates, quotes, tracking, JWT login, admin endpoints; domain events; STOMP push; audit log; health indicators	MySQL, MongoDB
Node/Express API (4000)	Enquiry and drawing-upload API; staff list/update of enquiries; catalogue cache; file logs; static API docs	MySQL, file storage
MySQL/MariaDB (3306)	Single shared database `pslaser`	—
MongoDB (27017)	Schema-less audit events (`activity_log`)	—
File storage	Private drawing uploads and request/event logs on disk	—

Key design decisions. (1) *Contract-first*: `services/CONTRACT.md` fixes the schema, JWT claims, price formula and API shapes. (2) *Shared schema*: PHP, Laravel and the servlet application use MySQL directly, which keeps each syllabus unit self-contained at the cost of repeating business rules. (3) *Graceful degradation*: the public Next.js estimator and the React portal keep working with labelled fallbacks when

Spring is unreachable. (4) *One rounding rule*: each cost line is rounded half-up to two decimals, then added.

8. Architecture Diagram

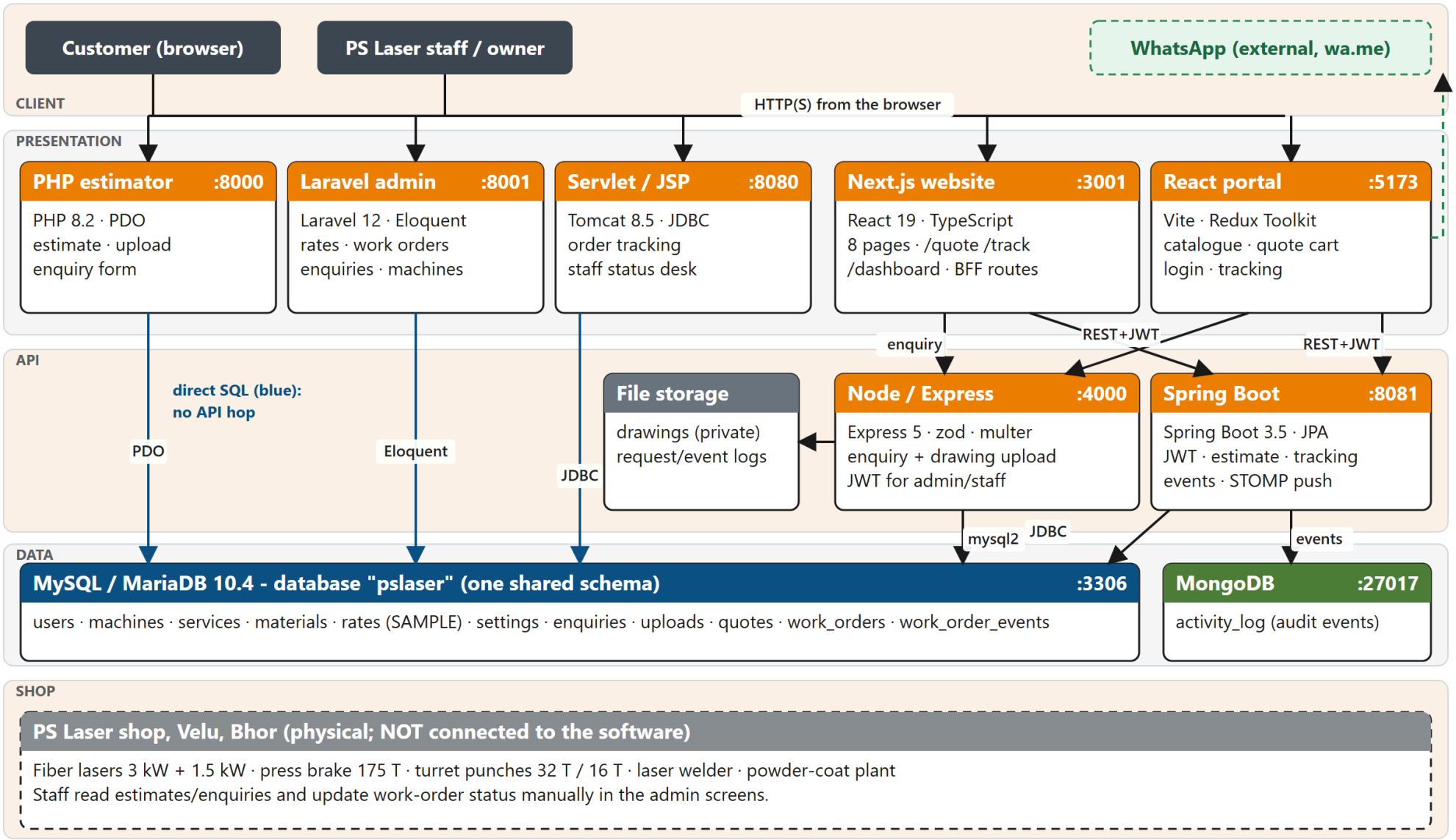


Figure 1. PS Laser system architecture as built and run locally (ports as run in October 2026).

How to read Figure 1. The customer and staff use a browser, which reaches any of the five front ends over HTTP(S). The Next.js website and React portal call the Spring Boot API (REST with a JWT) and the Node.js API (enquiries and uploads). The PHP estimator, Laravel admin and Servlet/JSP desk skip the API tier and use SQL directly against the shared MySQL schema (blue arrows). Spring and Node both write to MySQL; Spring also writes audit events to MongoDB and pushes live status on a STOMP topic; Node stores drawings and logs on disk. The dashed green arrow is the WhatsApp deep link used by every "Get Quote" button. The physical shop is dashed because it is not connected to the software. A larger version with a legend and cross-cutting security notes is supplied as `fig-architecture.png`.

9. Data Flow and Workflow

9.1 Main operational workflow

1. A customer opens the website, chooses a material and enters the part details; the estimator returns an indicative price breakdown.
2. The customer taps **Get Quote**, which opens WhatsApp with a pre-filled message, or submits the enquiry form with an optional drawing.
3. Node stores the enquiry and drawing and returns a WhatsApp link; staff see the enquiry in the Laravel, Node or Spring admin views.
4. After the shop agrees the real price outside the system, staff create a work order and move it through received → cutting → bending → welding → coating → inspection → dispatched.
5. Each status change adds an event row; the customer sees the timeline on /track (or the JSP page) by entering order number and phone; staff see live updates on the dashboard.

9.2 Key data flows

Table 10. Data flows

Flow	Path
Estimate	Browser → Next.js `/api/estimate` → Spring `POST /api/quotes/estimate` → reads `materials`, `rates`, `settings` in MySQL → breakdown JSON → page. If Spring is down the TypeScript formula answers and the page says "Offline estimate (sample)".
Tracking	Browser → `/track?orderNo=&phone=` (server-rendered) → Spring `GET /api/work-orders/track` → `work_orders` + `work_order_events` → timeline.
Status change	Staff action (Laravel, JSP or Spring admin) → update `work_orders` and insert `work_order_events` → Spring publishes `/topic/work-orders` → dashboard.
Enquiry	Next.js server action or React portal → Node `POST /api/enquiries` (JSON or multipart) → `enquiries` and `uploads` rows, private file on disk → `{id, whatsappUrl}`.
Login	Spring `POST /api/auth/login` → JWT (8 h); Node accepts the same token; Laravel and JSP keep their own sessions against the same `users` table.

9.3 Estimate formula (identical in PHP, Spring, JSP and Next.js)

```
areaM2   = lengthMm x widthMm / 1e6
weightKg = areaM2 x thicknessMm x density_g_cm3
material = weightKg x material_rate_per_kg
cutting  = (cutLengthMm / 1000) x cut_rate_per_m
piercing = pierces x pierce_rate
bending  = bends x settings.bend_rate_inr
coating  = powderCoat ? areaM2 x 2 x settings.powder_coat_rate_per_sqm : 0
perPart  = sum of the five lines, each rounded half-up to 2 dp
subtotal = round2(perPart x quantity)
gst      = round2(subtotal x settings.gst_percent / 100)
total    = subtotal + gst
```

Worked check [V]. Mild steel 2 mm, 500 × 300 mm, cut 1600 mm, 4 pierces, 2 bends, powder-coated, quantity 10, with the sample rates (75.00 INR/kg, 10.00 INR/m, 1.00 per pierce, 8 per bend, 120 per m², 18

% GST) gives per part 248.63, subtotal 2,486.30, GST 447.53 and **total INR 2,933.83**. This was returned by the Spring API on 16 Oct 2026 and by Spring, Next.js, PHP and the Java servlet application on 14 Oct 2026.

10. Interfaces and Dependencies

10.1 Spring Boot REST API (base http://localhost:8081)

Table 11. Spring endpoints

Method and path	Access	Purpose
`POST /api/auth/login`, `/api/auth/register`	Public	Issue JWT; create customer
`GET /api/machines`, `/api/machines/{slug}`, `/api/services`, `/api/materials`	Public	Catalogue
`POST /api/quotes/estimate`	Public	Calculate (not saved)
`POST /api/quotes`	Public	Calculate and save; returns quote number
`GET /api/work-orders/track?orderNo=&phone=`	Public	Order tracking
`GET /api/me`, `/api/me/quotes`	JWT	Own profile and quotes
`GET/POST/PUT /api/admin/work-orders`, `POST …/{id}/events`	JWT admin/staff	Manage orders
`GET /api/admin/enquiries`, `/api/admin/activity`	JWT admin/staff	Enquiries; MongoDB activity log
`/actuator/health`, `/actuator/info`, `/actuator/metrics`	Public	Monitoring

Errors use one shape: `{status, error, message, fieldErrors?}`. The token is sent as `Authorization: Bearer <token>`.

10.2 Node/Express API (base http://localhost:4000)

- `GET /api/health` returns `{ok, db}`; `GET /api/catalogue` returns machines and services (cached).
- `POST /api/enquiries` accepts JSON or multipart with an optional `drawing` file and returns `201 {id, whatsappUrl}`.
- `GET /api/enquiries` and `PATCH /api/enquiries/:id` require a JWT with role admin or staff.
- `GET /` serves the API documentation page.

10.3 Other interfaces

- **Next.js route handlers:** `POST /api/estimate`, `POST /api/auth/login`, `POST /api/auth/logout`, `GET /api/work-orders`.
- **WebSocket/STOMP:** topic `/topic/work-orders` published by Spring.
- **WhatsApp:** `https://wa.me/917719976990?text=…` (external; no server needed).
- **CORS:** allowed origins are http://localhost:3001, :5173, :8000, :8001 and :8080.

10.4 External dependencies

- WhatsApp (wa.me links) — external service; if unavailable the quote can still be requested by telephone.
- CDN assets for Bootstrap and jQuery (PHP estimator) and web fonts — need internet access.
- Package registries (npm, Maven Central, Packagist) — needed only for first-time installs.
- No hardware, payment, e-mail or SMS dependency exists.

11. Installation and Deployment Requirements

11.1 Preparation

- Free at least 5 GB on the drive that holds the project [P] (the handbook advises keeping 2 GB free before installing).
- Install Node.js 24, JDK 21 and XAMPP (PHP 8.2, MariaDB, Tomcat 8.5); set `JAVA_HOME` to the JDK folder.
- Load the database once (this **erases and recreates** `pslaser`): `mysql -uroot --default-character-set=utf8mb4 < services/db/schema.sql`, then `mysql -uroot --default-character-set=utf8mb4 pslaser < services/db/seed.sql`.
- Admin credentials for local testing are in `services/db/SEED_CREDENTIALS.md` (test values only; not reproduced here).

11.2 Start order and success checks [V]

Table 12. Start commands (run from the folder shown)

#	Service	Command	Success check
1	MySQL	`C:\xampp\mysql_start.bat`	`mysql -uroot -e "SELECT 1"` prints 1
2	Spring (services\spring-api)	`.\mvnw.cmd spring-boot:run "-Dspring-boot.run.profiles=dev,mongo-embedded"`	`/actuator/health` → UP
3	Node (services\node-api)	`npm start`	`/api/health` → `{"ok":true,"db":true}`
4	Website (project root)	`npm run dev -- -p 3001`	`/` → 200
5	PHP	`C:\xampp\php\php.exe -S localhost:8000 -t services/php-estimator/public`	`/` → 200
6	Laravel (services\laravel-admin)	`C:\xampp\php\php.exe artisan serve --port=8001`	`/login` → 200
7	Tomcat	`C:\xampp\tomcat\bin\startup.bat` (JAVA_HOME set)	`/pslaser/` → 200
8	React portal (services\react-portal)	`npm run dev`	Page loads on 5173

Use the `dev,mongo-embedded` Spring profile when Docker is not running. To stop: Ctrl+C in each terminal, `C:\xampp\tomcat\bin\shutdown.bat` for Tomcat, and `C:\xampp\mysql_stop.bat` for MySQL last. Test commands: `npm test` (Node), `php artisan test` (Laravel), `mvnw.cmd test` (Spring), `npm run check` (website).

11.3 Production deployment (proposed, not done) [P]

- Replace the development database credentials and the development `JWT_SECRET` with secrets from a secrets manager.
- Serve every service behind HTTPS on a real domain; set the site's `metadataBase`, sitemap and robots rules.
- Use a managed MySQL with automated backups; use a persistent MongoDB (not the embedded instance).
- Containerise the services (docker-compose) and add a CI pipeline that runs the automated tests.

12. Assumptions, Constraints and Future Enhancements

12.1 Assumptions

- The shop owner will supply real rates before any customer relies on an estimate.
- Final prices are agreed on WhatsApp or by telephone; the website only gives an indication.
- Staff update job status manually and have access to a browser.
- Customers know the order number and the phone number used when the order was created.

12.2 Constraints and known limitations

- Rate card is **sample data**; product and gallery pictures are representative renders, not photographs of finished PS Laser jobs; demo orders `PSL-DEMO-001/002` are fake test data.
- Runs locally only; no domain, HTTPS or public deployment. Development secrets must not be used publicly.
- No machine integration, payment, e-mail or SMS; enquiry delivery is a database row plus a WhatsApp link.
- Embedded MongoDB loses its data on restart; Docker was not available during testing.
- Not tested: visual responsive pass, every image, React portal → Node → database enquiry path, live push to the dashboard, upload re-test with real files, load and performance.
- Some engineering text on the site (for example which laser suits thick plate) is inferred and awaits owner confirmation.

12.3 Future enhancements

1. Enter the real rate card and add a "valid until" date to quotes.
2. Send e-mail or WhatsApp Business notifications when a job status changes.
3. Parse DXF drawings to propose cut length and pierce counts automatically.
4. Add end-to-end browser tests (Playwright) and a one-command start script or docker-compose file.
5. Deploy to a managed host with HTTPS, backups and monitoring.
6. Add owner-supplied photographs and, only if genuine, testimonials and a client list.

References

1. *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, ISO/IEC/IEEE 29148:2018, 2018.
2. *Systems and Software Engineering — SQuaRE — System and Software Quality Models*, ISO/IEC 25010:2011, 2011.
3. M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Internet Eng. Task Force, RFC 7519, May 2015.
4. R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. California, Irvine, CA, USA, 2000.
5. *Spring Boot Reference Documentation*, version 3.5. [Online]. Available: https://docs.spring.io/spring-boot/
6. *Next.js Documentation*, Vercel Inc. [Online]. Available: https://nextjs.org/docs
7. *Laravel Documentation*, version 12.x. [Online]. Available: https://laravel.com/docs/12.x